

Defense Document: Multi-Temporal Deforestation Segmentation

With Sentinel-2 and Global Forest Watch Labels

Persiapan Sidang / Presentasi Lomba

Deforest.id Research Team

Contents

1	Executive Summary (Apa yang Kita Buat?)	3
2	Problem Statement: Kenapa Ini Perlu Dibuat?	3
2.1	Masalah Deforestasi di Indonesia	3
2.2	Kesenjangan (Research Gap)	3
2.3	Pertanyaan Riset	3
3	Dataset: Dari Mana Datanya, Kenapa?	3
3.1	Mengapa Sentinel-2, Bukan Landsat?	3
3.2	Mengapa 5 Band Input (R, G, B, NIR, NDVI)?	4
3.3	Mengapa Cloud-Free Composite, Bukan Single Scene?	4
3.4	Mengapa 64×64 Pixel (0.64 ha)?	4
3.5	Mengapa GFW Label?	4
3.6	Dataset Statistics	5
3.7	Pertanyaan Potensial di Sidang	5
4	Deep Dive: Konsep Fundamental	5
4.1	Apa Itu Image Segmentation?	5
4.2	Bagaimana Cara Kerja U-Net?	5
4.3	Bagaimana Cara Kerja ResNet (Residual Network)?	6
4.4	Bagaimana Cara Kerja Stacked Multi-Temporal U-Net?	6
4.5	Bagaimana Cara Kerja Siamese FC-Siam-diff?	6
4.6	NIR (Near-Infrared): Kenapa Penting?	7
4.7	Hansen Global Forest Change (GFC): Apa Itu Labelnya?	7
4.8	Metric: IoU (& mIoU)	8
4.9	Metric: F1 Score	8
4.10	Loss Function: Dice Loss	9
4.11	Loss Function: Tversky Loss	9
4.12	Loss Function: Focal Loss	9
4.13	Loss Function: Lovász-Softmax	10
4.14	Ringkasan Loss Functions	10
5	Arsitektur: Kenapa Tiga Model?	10
5.1	Pertanyaan Fundamental	10
5.2	Model 1: Single-Date (Baseline)	10
5.3	Model 2: Stacked Multi-Temporal	10
5.4	Model 3: Siamese FC-Siam-diff	11
5.5	Mengapa Bukan Model Lain?	11
5.6	Pertanyaan Potensial di Sidang	11
6	Training: Semua Detail + Alasannya	11

6.1	Funksi Loss: Kenapa Dice?	11
6.2	Optimizer: Kenapa AdamW?	12
6.3	Learning Rate Scheduling: Kenapa Cosine Annealing?	12
6.4	Gradient Accumulation: Kenapa Accum=2?	12
6.5	Oversampling: Kenapa 10×?	12
6.6	Augmentasi: Kenapa Minimal?	13
6.7	AMP (Mixed Precision): Kenapa?	13
6.8	Gradient Clipping: Kenapa 1.0?	13
6.9	RAM Caching: Kenapa?	13
7	Inference & Evaluation	13
7.1	Test-Time Augmentation (TTA)	13
7.2	Global vs Per-Sample Metrics	14
7.3	Mengapa Threshold 0.5?	14
7.4	Pertanyaan Potensial	14
8	Hasil: Interpretasi Lengkap	14
8.1	Tabel Hasil	14
8.2	Analisis Per Model	14
8.3	Mengapa IoU Hanya 0.137?	15
8.4	Perbandingan dengan ICP Paper (NDVI pseudo-label)	15
8.5	Overfitting: Analisis	15
9	Pipeline: Gambaran Sistem End-to-End	16
10	Pertanyaan Sulit yang Mungkin Muncul	16
10.1	Tentang Relevansi dan Kontribusi	16
10.2	Tentang Metode	16
10.3	Tentang Evaluasi	17
10.4	Tentang Aplikasi	17
11	Limitations: Jawaban Jujur	17
12	Future Work: Arah Pengembangan	18
13	Quick Reference Cards	18

1 Executive Summary (Apa yang Kita Buat?)

Kami membangun **sistem deteksi deforestasi otomatis** menggunakan deep learning pada citra satelit Sentinel-2 (resolusi 10 m). Sistem ini menggunakan **label dari Global Forest Watch** (GFW, resolusi 30 m) sebagai target training — tanpa label manual.

Kami membandingkan **tiga arsitektur**:

1. **Single-date baseline**: U-Net sederhana, hanya melihat citra setelah deforestasi (5 channel). Skor: test IoU 0.122.
2. **Stacked multi-temporal**: Pre-event + post-event digabung jadi 10 channel input. Skor: test IoU 0.119.
3. **Siamese FC-Siam-diff**: Dua encoder identik (weight sharing) untuk pre dan post, decoder menghitung perbedaan fitur secara eksplisit. Skor: **test IoU 0.137**.

Kesimpulan utama: Cara kita menggabungkan informasi temporal **lebih penting** daripada apakah kita menggunakan informasi temporal atau tidak. Siamese architecture menang karena explicit change features.

2 Problem Statement: Kenapa Ini Perlu Dibuat?

2.1 Masalah Deforestasi di Indonesia

Indonesia adalah negara dengan **hutan tropis terluas ketiga** di dunia (~94 juta hektar). Setiap tahun, sekitar 300.000 hektar hutan alam hilang. Dari jumlah ini, sebagian besar adalah **small-holder clearing** — pembukaan lahan skala kecil oleh individu atau komunitas lokal.

Mengapa small-holder clearing sulit dideteksi?

- Luas lahan yang dibuka: 0.1 – 0.5 hektar (seukuran lapangan basket sampai lapangan sepak bola mini).
- Sistem monitoring eksisting (GFW, SIMONTANA) menggunakan citra Landsat 30 m. Satu pixel Landsat = 0.09 ha. Jadi clearing 0.1 ha hanya muat 1–2 pixel — tidak cukup untuk analisis shape atau konfirmasi.
- Dilakukan secara tersebar, bukan dalam blok besar.

2.2 Kesenjangan (Research Gap)

Sistem yang ada sekarang:

- **GFW**: 30 m, tahunan. Bagus untuk skala global, tapi tidak bisa deteksi sub-hektar.
- **SIMONTANA** (Indonesia): Minimum Mapping Unit 6.25 ha. Jelas terlalu besar.
- **Planet Labs** (3 m): Berbayar. Tidak feasible untuk monitoring kontinu di Indonesia.

Solusi yang kami usulkan: Gunakan **Sentinel-2** (10 m, gratis) + **deep learning** untuk mendeteksi deforestasi pada skala yang sebelumnya tidak terjangkau.

2.3 Pertanyaan Riset

1. Bisakah label GFW (30 m) digunakan untuk melatih model segmentasi pada Sentinel-2 (10 m)?
2. Apakah multi-temporal fusion lebih baik daripada single-date? Kalau iya, arsitektur fusion mana yang terbaik?
3. Seberapa besar dampak label noise terhadap performa?

3 Dataset: Dari Mana Datanya, Kenapa?

3.1 Mengapa Sentinel-2, Bukan Landsat?

Parameter	Sentinel-2	Landsat 8-9
Resolusi spasial (RGB, NIR)	10 m	30 m
Resolusi temporal	5 hari	16 hari
Luas minimum detectable	0.01 ha	0.09 ha
Biaya	Gratis	Gratis
Cakupan	Global	Global

Resolusi 10 m itu penting kenapa? Deforestasi skala kecil (0.1 ha) terdiri dari sekitar 100 pixel di Sentinel-2, tapi cuma ~11 pixel di Landsat. Dengan 100 pixel, model bisa belajar *tekstur* dan *bentuk* — bukan cuma ada/tidak ada perubahan.

Temporal resolution 5 hari itu penting kenapa? Indonesia sering berawan. Dengan revisit 5 hari, kita punya banyak kesempatan mendapatkan scene bebas awan dalam satu tahun. Landsat 16 hari: kadang cuma 1–2 scene bersih per tahun.

3.2 Mengapa 5 Band Input (R, G, B, NIR, NDVI)?

Citra satelit itu bukan foto biasa. Setiap "band" adalah lapisan informasi spektral yang berbeda:

- **RGB (Blue 490nm, Green 560nm, Red 665nm):** Sama seperti mata manusia. Untuk visualisasi dan interpretasi visual. Juga menangkap informasi tanah terbuka (soil exposure setelah deforestasi).
- **NIR (Near-Infrared, 842nm):** Tidak terlihat oleh manusia. Vegetasi sehat memantulkan NIR sangat kuat ($\sim 40\text{--}50\%$). Tanah dan air memantulkan NIR rendah ($\sim 10\text{--}20\%$). Inilah band paling penting: deforestasi menyebabkan penurunan NIR yang sangat mencolok.
- **NDVI (Normalized Difference Vegetation Index):** Rumusnya $(NIR - Red)/(NIR + Red)$. Hasilnya angka antara -1 sampai 1. Hutan lebat: 0.7–0.9. Tanah terbuka: 0.1–0.2. Air: negatif. NDVI memberikan informasi vegetasi yang **dinormalisasi** — tidak terpengaruh oleh kecerahan absolut citra (bayangan, sudut matahari, dll).

Mengapa tidak 10 band (semua band Sentinel-2)? Sentinel-2 punya 13 band. Band 1 (60m), 2-4 (10m), 5-7 (20m), 8 (10m), 8a (20m), 9-10 (60m), 11-12 (20m). Band 20 m perlu upsampling ke 10 m. Ini:

- Menambah ukuran data $2\times$ tanpa informasi baru (resolusi asli sudah 20m).
- Menambah noise dari interpolasi.
- Uji coba awal dengan 10 band menunjukkan performa setara dengan 5 band — jadi tidak ada gunanya.

3.3 Mengapa Cloud-Free Composite, Bukan Single Scene?

Single scene: Satu gambar pada satu tanggal. Masalah:

- Awan. Indonesia tropis, awan hampir selalu ada.
- Bayangan awan. Bisa mirip deforestasi secara spektral.
- Haze (kabut asap). Sering terjadi akibat kebakaran hutan.

Cloud-free composite: Ambil semua scene dalam 1 tahun, masking awan, lalu rata-ratakan median/sentinel yang bersih. Hasilnya: permukaan bumi yang bersih tanpa awan. **Trade-off:** Deforestasi yang terjadi di pertengahan tahun bisa ter-blur karena tercampur dengan citra sebelum dan sesudah deforestasi. **Kami menerima trade-off ini** karena model yang dilatih dengan data berawan akan belajar mendeteksi awan, bukan deforestasi.

3.4 Mengapa 64×64 Pixel (0.64 ha)?

Ini adalah keputusan yang sangat penting. Mari kita analisis:

- **GPU Memory:** RTX 3060 punya 12GB VRAM. Dengan 64×64 , batch size 64 muat. Kalau 128×128 , batch size maksimal cuma 16 — terlalu kecil untuk training stabil.
- **Receptive Field:** U-Net dengan 4 encoder block (masing-masing stride 2) punya teoritical receptive field 140×140 . Artinya, output di satu pixel sebenarnya "melihat" area 140×140 di input. Ini lebih besar dari chip 64×64 , jadi konteksnya cukup.
- **Small Clearing:** Deforestasi 0.1 ha = 10×10 pixel di Sentinel-2. Chip 64×64 muat untuk mendeteksi ini dengan konteks sekelilingnya.
- **Jumlah Sampel:** 64×64 menghasilkan lebih banyak chip per scene daripada 128×128 . Kita dapatkan 23,135 training chips.

Mengapa tidak lebih kecil (32×32)? Terlalu kecil: tidak cukup konteks untuk membedakan deforestasi dari non-forest. Boundary informasi hilang.

3.5 Mengapa GFW Label?

Ini adalah **pertanyaan paling kritis**. Kenapa kita tidak pakai label yang bagus?

Realitas: Membuat label deforestasi manual untuk ribuan hektar hutan Indonesia membutuhkan:

- Tenaga annotator terlatih (bisa membedakan deforestasi dari logging, kebakaran, pertanian).
- Waktu berbulan-bulan.
- Biaya puluhan juta rupiah.

GFW (Global Forest Watch) [1] menyediakan *lossyear* layer: peta tahunan kehilangan tutupan pohon dari tahun 2000–2023. Ini adalah **satu-satunya dataset deforestasi open-access yang mencakup Indonesia**.

Tapi GFW punya masalah:

- Berasal dari Landsat (30 m). Jadi 1 pixel GFW = 9 pixel Sentinel-2.
- Ini menciptakan **label noise struktural**: batas deforestasi di GFW tidak presisi di resolusi 10 m.
- Ada juga **temporal noise**: GFW melaporkan perubahan tahunan, sementara komposit kami spesifik ke 2020 dan 2023. Deforestasi yang terjadi 2021 atau 2022 tercatat sebagai "perubahan" tapi timing-nya tidak tepat.

Mengapa tetap pakai GFW? Karena ini adalah *proof of concept*. Tujuan kami bukan mencapai akurasi maksimal, tapi **membuktikan pipeline end-to-end bisa jalan**. Kalau pipeline ini berhasil dengan label noise tinggi, maka dengan label yang lebih baik (manual, resolusi tinggi), hasilnya pasti lebih baik.

3.6 Dataset Statistics

- **Sumber data**: 13 scene Sentinel-2 mencakup hutan lindung di Riau dan Kalimantan Tengah.
- **Total paired chips (pre+post)**: 40,484.
- **Train / Val / Test**: 23,135 / 5,822 / 11,527.
- **Global positive pixel ratio**: 0.38%.
- **Chips dengan zero deforestation**: 74% (30,000 chips dari 40,484).
- **Perbandingan train vs test**: Test set punya density deforestasi lebih tinggi (0.9% vs 0.38%) — ini penting nanti untuk interpretasi hasil.

3.7 Pertanyaan Potensial di Sidang

- **Q**: *Kenapa train-test split-nya tidak random? Kenapa dipisah per scene?*
- **A**: Kalau random, chips dari scene yang sama bisa masuk train dan test. Model akan "menghafal" karakteristik scene itu. Dengan split geografis, kita menguji generalization: apakah model bisa mendeteksi deforestasi di lokasi yang belum pernah dilihat? Ini lebih realistis.
- **Q**: *13 scene itu seberapa representatif untuk Indonesia?*
- **A**: Terbatas. Riau dan Kalteng adalah hot-spot deforestasi, tapi tipe hutan di Papua, Kalimantan Utara, Sulawesi berbeda. Generalizability masih perlu diuji.

4 Deep Dive: Konsep Fundamental

4.1 Apa Itu Image Segmentation?

Image segmentation = memberi label ke **setiap pixel** dalam gambar. Bukan klasifikasi gambar ("apakah foto ini ada hutan?"), tapi klasifikasi per-pixel ("pixel mana yang merupakan hutan? pixel mana yang merupakan deforestasi?").

Output model: Untuk setiap pixel, model mengeluarkan probabilitas antara 0 dan 1. Probabilitas > 0.5 = deforestasi. Hasil akhir: binary mask ukuran sama dengan input.

Kenapa segmentasi, bukan deteksi objek (bounding box)? Deforestasi tidak punya bentuk yang tetap — bisa persegi (ladang), memanjang (jalan), atau irregular. Bounding box tidak cukup presisi.

4.2 Bagaimana Cara Kerja U-Net?

Analogi: Bayangkan seorang pelukis yang ingin meniru sebuah lukisan. Pertama, dia mengamati lukisan dari jauh untuk memahami komposisi global (encoder). Lalu, dia melukis detailnya dari memori, sesekali melihat lukisan asli untuk memastikan detail kecil — posisi mata, lipatan kain (skip connections).

Struktur U-Net [2]:

- **Encoder (sisi kiri)**: Jalan turun. Terdiri dari **convolutional blocks** (Conv2D + BatchNorm + ReLU) diikuti **max pooling** (2×2 , stride 2) yang mengurangi resolusi setengahnya.
 - Input: $64 \times 64 \times 5$ (5 channel)
 - Level 1: $64 \times 64 \times 64$ (64 filter)
 - Level 2: $32 \times 32 \times 128$
 - Level 3: $16 \times 16 \times 256$
 - Level 4: $8 \times 8 \times 512$
 - Bottleneck: $4 \times 4 \times 1024$
- **Bottleneck**: Resolusi paling kecil, jumlah channel paling banyak. Di sinilah model "memahami" gambar secara global. Setiap pixel di bottleneck "melihat" area 140×140 pixel di input (receptive field).

- **Decoder (sisi kanan):** Jalan naik. **Transposed convolution** (upsampling $2\times$) mengembalikan resolusi. Setiap upsampling diikuti concatenation dengan feature map dari encoder level yang sama (skip connection), lalu convolution.
- **Skip connections:** Ini yang membuat U-Net powerful. Decoder tidak hanya menggunakan informasi dari bottleneck (yang sudah sangat abstrak, resolusinya 4×4), tapi juga menerima feature maps detail dari encoder (yang masih memiliki informasi posisi presisi). Tanpa skip connections, decoder akan menghasilkan output yang kasar dan boundary-nya tidak presisi.
- **Output:** $64\times 64\times 1$ (satu channel), setiap pixel berisi probabilitas $[0, 1]$.

Mengapa U-Net cocok untuk remote sensing? Karena:

- Skip connections mempertahankan detail batas (boundary) — penting untuk delineasi deforestasi.
- Bisa bekerja dengan jumlah data terbatas (ribuan, bukan jutaan).
- Bobot encoder bisa ditransfer dari model pretrained (ResNet, EfficientNet).

4.3 Bagaimana Cara Kerja ResNet (Residual Network)?

Masalah: Semakin dalam jaringan, gradient harus melewati semakin banyak layer. Gradient bisa menjadi sangat kecil (vanishing) atau sangat besar (exploding). Akibatnya: jaringan 50 layer bisa lebih buruk daripada 20 layer.

Solusi ResNet [3]: Residual learning.

- **Ide:** Alih-alih belajar fungsi $H(x)$ secara langsung, belajar residual $F(x) = H(x) - x$. Output aktual: $H(x) = F(x) + x$.
- **Residual block:** $x \rightarrow [\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{BN}] \rightarrow +x$ (shortcut) $\rightarrow \text{ReLU}$.
- **Shortcut connection:** Menambahkan input x langsung ke output convolution. Gradient bisa mengalir melalui shortcut ini tanpa modifikasi — sehingga gradient tidak vanishing meski di jaringan sangat dalam.
- **ResNet34:** 34 layer, 21.3M parameter. Terdiri dari 4 stages dengan jumlah channel $[64, 128, 256, 512]$.

Mengapa pretrained ImageNet membantu? ImageNet berisi 14 juta gambar natural (kucing, mobil, dll.). Layer pertama ResNet belajar mendeteksi *low-level features* universal: tepi, sudut, gradien, tekstur. Features ini juga berguna untuk citra satelit. Layer tinggi (yang mendeteksi objek spesifik) akan diadaptasi selama fine-tuning. Ini mempercepat training dan meningkatkan performa dibanding training dari random weights.

4.4 Bagaimana Cara Kerja Stacked Multi-Temporal U-Net?

Stacked = ditumpuk. Pre-event (5 band) dan post-event (5 band) digabungkan secara **channel-wise concatenation** menjadi 10 channel.

- Input: $64\times 64\times 10$.
- Channel 1-5: pre-event (B, G, R, NIR, NDVI).
- Channel 6-10: post-event (B, G, R, NIR, NDVI).
- Arsitektur: sama persis dengan U-Net biasa, hanya input layer pertama disesuaikan dari $3 \rightarrow 10$ channel.

Mengapa ini disebut “naive fusion”? Karena model menerima semua informasi tanpa struktur. Model harus belajar sendiri bahwa:

1. Channel 1-5 berkorelasi dengan channel 6-10 (pre/post).
2. Perbedaan antara channel 1-5 dan 6-10 itu signifikan.

Ini tugas tambahan yang tidak perlu — Siamese memberikan struktur ini secara eksplisit.

Mengapa adaptasi layer pertama perlu hati-hati? ResNet34 pretrained menerima 3 channel (RGB). Input kita 10 channel. Cara adaptasi: bobot untuk 3 channel pertama = pretrained RGB. Bobot 7 channel sisanya diinisialisasi random (atau copy dari RGB). Tanpa adaptasi yang benar, pretrained weights menjadi tidak berguna.

4.5 Bagaimana Cara Kerja Siamese FC-Siam-diff?

Siamese artinya “kembar” (berasal dari bahasa Siam/ Thailand, digunakan untuk menggambarkan kembar siam — dua tubuh bergabung).

Arsitektur FC-Siam-diff [4]:

- **Dua encoder identik** (ResNet34) dengan **shared weights**. Encoder A memproses pre-event, Encoder B memproses post-event. Karena bobotnya sama, keduanya mengekstrak fitur dengan cara yang persis sama.
- **Feature differencing:** Di setiap level encoder, kita punya f_{pre} dan f_{post} . Decoder menerima **tiga** input:

$$[f_{post}, f_{pre}, f_{post} - f_{pre}]$$

- **Kenapa tiga input?**
 - f_{post} : Informasi post-event (untuk deteksi deforestasi secara langsung).
 - f_{pre} : Informasi pre-event (konteks: area ini dulunya hutan atau bukan?).
 - $f_{post} - f_{pre}$: **Perubahan eksplisit**. Nilai positif = fitur baru muncul. Nilai negatif = fitur menghilang. Deforestasi: nilai negatif besar di channel NIR/NDVI.
- **Decoder:** Concatenate ketiga input, lalu konvolusi untuk menghasilkan feature map yang lebih kaya. Sama seperti U-Net, ada skip connections dari encoder ke decoder.
- **Output:** $64 \times 64 \times 1$.

Mengapa weight sharing penting? Tanpa weight sharing:

- Encoder A dan B bisa mempelajari representasi berbeda.
- Model bisa “curang”: satu encoder fokus ke warna merah, satu lagi ke NIR. Perbedaan yang terdeteksi bukan karena perubahan tutupan lahan, tapi karena encoder berbeda.
- Weight sharing memastikan: *jika inputnya sama, output encodernya harus sama*. Ini memberikan konsistensi.

Mengapa $f_{post} - f_{pre}$ lebih baik dari concatenation saja?

- Concatenation: decoder menerima 10 channel (5 pre + 5 post). Tidak tahu mana pre/post. Harus belajar relasi antar channel — tugas implisit.
- Difference: decoder langsung diberi tahu “channel ini adalah perubahannya”. Tidak perlu belajar korelasi antar channel — perubahan sudah dihitung.
- Hasil: model lebih cepat konvergen, lebih robust terhadap noise.

Visual langkah-langkah:

1. Pre (5 band) $\rightarrow$ Encoder $\rightarrow f_{pre}$ (multi-level).
2. Post (5 band) $\rightarrow$ Encoder (weight sama) $\rightarrow f_{post}$ (multi-level).
3. Di setiap level: hitung $f_{post} - f_{pre}$.
4. Decoder menerima $[f_{post}, f_{pre}, f_{post} - f_{pre}]$.
5. Decoder upsample, skip connection dari encoder, konvolusi.
6. Output: mask deforestasi 64×64 .

4.6 NIR (Near-Infrared): Kenapa Penting?

NIR adalah kunci deteksi deforestasi.

Cahaya tampak (RGB) = panjang gelombang 400-700nm. Near-Infrared (NIR) = 700-1100nm. Tidak terlihat oleh mata manusia.

Mengapa NIR penting untuk vegetasi?

- Daun sehat memiliki struktur sel mesofil yang memantulkan NIR sangat kuat ($\sim 40-50\%$ dari energi NIR yang datang).
- Ini disebut **red-edge effect**: perbedaan drastis antara refleksi Red (5-10%) dan NIR (40-50%).
- Ketika pohon ditebang: struktur sel daun hilang. Refleksi NIR turun drastis ke $\sim 10-15\%$.
- Tanah telanjang: refleksi NIR rendah.
- Air: refleksi NIR hampir nol.

Contoh numerik:

- Hutan lebat: Red=0.05, NIR=0.45. $NDVI = (0.45-0.05)/(0.45+0.05) = 0.80$.
- Tanah terbuka: Red=0.15, NIR=0.20. $NDVI = (0.20-0.15)/(0.20+0.15) = 0.14$.
- Deforestasi: Red naik, NIR turun. NDVI turun drastis.

Band 8 Sentinel-2 adalah NIR pada 842nm, resolusi 10m. Ini adalah band paling informatif untuk deteksi vegetasi.

4.7 Hansen Global Forest Change (GFC): Apa Itu Labelnya?

Hansen GFC [1] = dataset global pertama yang memetakan perubahan tutupan pohon dari tahun 2000-2023 pada resolusi 30m. Dipublikasikan oleh **Matthew Hansen** dan tim di University of Maryland.

Apa isi datasetnya?

- **Treecover2000:** Persentase tutupan pohon per pixel pada tahun 2000 (0-100%).
- **Loss:** Tahun kehilangan tutupan pohon (2001-2023). 0 = tidak ada kehilangan.
- **Gain:** Tahun gain tutupan pohon (2001-2012).
- **Mask:** Mask data (air, tidak ada data).

Bagaimana label dibuat?

1. Ribuan citra Landsat 30m diproses per tahun.

2. Algoritma random forest mendeteksi perubahan reflektansi yang konsisten dengan kehilangan tutupan pohon.
3. Validasi manual: ribuan plot diverifikasi oleh annotator.
4. Dataset diperbarui tahunan (GFW).

Mengapa disebut GFC bukan GFW?

- **Hansen GFC** = dataset asli dari publikasi 2013.
- **GFW (Global Forest Watch)** = platform yang mendistribusikan data ini secara operasional.
- Dalam keseharian, orang sering menyebutnya “GFW label” atau “Hansen label”. Keduanya merujuk ke data yang sama.

Keterbatasan untuk proyek kami:

- Resolusi 30m. 1 pixel GFC = 9 pixel Sentinel-2.
- Threshold “tree cover” GFC: tutupan pohon > 30%. Deforestasi ringan (penjarangan) tidak terdeteksi.
- Temporal: tahunan. Deforestasi bulan Maret dan Desember di tahun yang sama tidak bisa dibedakan.
- *Commission error*: Beberapa perubahan non-forest (kebakaran, pertanian rotasi) terdeteksi sebagai loss.

4.8 Metric: IoU (& mIoU)

IoU = Intersection over Union.

$$IoU = \frac{|A \cap B|}{|A \cup B|} = \frac{TP}{TP + FP + FN}$$

Penjelasan visual: Bayangkan dua lingkaran:

- Lingkaran A: area deforestasi sebenarnya (ground truth).
- Lingkaran B: area deforestasi yang diprediksi model.
- **Intersection (irisan):** Area yang benar-benar deforestasi dan diprediksi dengan benar = TP.
- **Union (gabungan):** Total area yang merupakan deforestasi (TP + FP + FN).
- IoU = irisan / gabungan.

Nilai IoU:

- IoU = 1: Prediksi sempurna (irisan = gabungan).
- IoU = 0: Tidak ada overlap (prediksi dan label tidak bersentuhan).
- IoU > 0.5: Dianggap bagus untuk segmentasi objek kecil.
- IoU > 0.7: Sangat bagus (state-of-the-art untuk objek besar seperti mobil di Cityscapes).

Global IoU vs Mean IoU:

- **Global IoU:** Hitung TP, FP, FN dari *semua* pixel di semua chips, baru hitung IoU. Tidak terpengaruh oleh chips tanpa deforestasi.
- **Mean IoU (mIoU):** Hitung IoU per-chip, lalu rata-rata. 74% chips IoU=0, jadi mIoU sangat rendah (~0.013).
- Di paper kami: laporkan global IoU.

mIoU (mean IoU) untuk multi-class: Rata-rata IoU dari semua kelas. Kalau 2 kelas (background + deforestasi):

$$mIoU = \frac{IoU_{bg} + IoU_{deforest}}{2}$$

Tapi untuk tugas kami, IoU background selalu tinggi (>0.98), jadi mIoU tidak informatif. Kami fokus ke IoU deforestasi.

4.9 Metric: F1 Score

F1 = Harmonic mean of Precision and Recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Precision: Dari semua pixel yang kita prediksi sebagai deforestasi, berapa persen yang benar?

$$Precision = \frac{TP}{TP + FP}$$

Recall: Dari semua pixel deforestasi sebenarnya, berapa persen yang berhasil kita deteksi?

$$Recall = \frac{TP}{TP + FN}$$

Kenapa F1, bukan accuracy?

- Accuracy: $\frac{TP+TN}{TP+TN+FP+FN}$.
- Dengan 99.62% background, prediksi semua 0 = accuracy 99.62% — tapi model tidak berguna.
- F1 mengabaikan TN. Fokus ke performa pada kelas minoritas (deforestasi).

Interpretasi F1:

- F1 = 1: precision dan recall sempurna.
- F1 = 0: salah satu precision atau recall = 0.
- F1 $\sim$ 0.24 (Siamese): precision 0.196, recall 0.311. Trade-off: model lebih cenderung mendeteksi (recall lebih tinggi) dengan konsekuensi false positive (precision lebih rendah).

4.10 Loss Function: Dice Loss

Rumus:

$$DiceLoss = 1 - \frac{2 \times TP + \epsilon}{2 \times TP + FP + FN + \epsilon}$$

di mana $\epsilon = 1.0$ (smoothing factor, mencegah division by zero).

Kenapa TP dihitung 2×? Ini membuat loss lebih sensitif terhadap false negatives (FN). Setiap pixel deforestasi yang terlewat dihitung 2× di penyebut. Model “dihukum” lebih berat kalau melewatkan deforestasi.

Perbandingan dengan IoU:

- IoU: $\frac{TP}{TP+FP+FN}$.
- Dice: $\frac{2 \times TP}{2 \times TP + FP + FN}$.
- Dice $\approx \frac{2 \times IoU}{1 + IoU}$. Transformasi monotonik — optimal di threshold yang sama.

Mengapa efektif untuk extreme imbalance?

- Cross-entropy: menghitung loss untuk *setiap pixel*. 99.62% pixel adalah background. Loss didominasi background. Model optimal = prediksi semua 0.
- Dice loss: hanya pixel yang diprediksi positif yang berkontribusi ke numerator (TP). Denominator juga hanya melibatkan TP, FP, FN. **TN tidak masuk hitungan**. Model tidak diuntungkan dengan memprediksi background.
- Analogi: ujian dengan 100 soal, 99 soal mudah, 1 soal sulit. Cross-entropy = nilai berdasarkan semua 100 soal. Dice = nilai hanya berdasarkan soal sulit itu.

4.11 Loss Function: Tversky Loss

Rumus:

$$TverskyLoss = 1 - \frac{TP + \epsilon}{TP + \alpha \cdot FP + \beta \cdot FN + \epsilon}$$

di mana $\alpha + \beta = 1$.

Perbedaan dengan Dice:

- Dice: FP dan FN diberi bobot sama.
- Tversky: FP dan FN bisa diberi bobot berbeda.
- α = bobot FP, β = bobot FN.

Kapan digunakan?

- $\alpha > \beta$: hukuman false positive lebih berat. Precision naik, recall turun.
- $\beta > \alpha$: hukuman false negative lebih berat. Recall naik, precision turun.

Hasil uji coba kami ($\alpha = 0.3, \beta = 0.7$):

- Recall naik dari 0.311 (Dice) ke 0.402 (Tversky).
- Tapi precision turun drastis dari 0.196 ke 0.089.
- F1 turun dari 0.240 ke 0.146.
- **Kesimpulan:** Untuk tugas ini, menekan FN terlalu keras menghasilkan terlalu banyak false positives. Dice (bobot seimbang) lebih baik.

4.12 Loss Function: Focal Loss

Rumus:

$$FocalLoss = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

di mana:

- p_t : probabilitas prediksi untuk kelas yang benar.
- γ : focusing parameter (biasanya 2).
- α_t : class weighting (biasanya inverse frequency).

Bagaimana cara kerjanya?

- Cross-entropy: $CE = -\log(p_t)$.
- Focal loss menambahkan $(1 - p_t)^\gamma$:
- Jika p_t tinggi (easy sample, model sudah yakin): faktor $(1 - p_t)^\gamma$ kecil $\rightarrow$ loss ditekan.
- Jika p_t rendah (hard sample, model tidak yakin): faktor $(1 - p_t)^\gamma$ besar $\rightarrow$ loss tetap tinggi.
- Efek: model fokus ke sample yang sulit, tidak membuang energi pada sample yang sudah bisa.

Kok bisa untuk imbalance?

- Background (easy samples): p_t tinggi, loss ditekan.
- Deforestasi (hard samples): p_t rendah, loss tetap.
- Tapi untuk extreme imbalance seperti kita (0.38%), focal loss belum cukup. γ perlu sangat tinggi atau α_t perlu dituning dengan hati-hati.

Hasil uji coba ($\gamma = 2, \alpha = 0.996$): Lebih buruk dari Dice. Kemungkinan karena tuning γ dan α yang sulit. Focal loss populer di detection (COCO) tapi kurang efektif untuk segmentation dengan imbalance ekstrem.

4.13 Loss Function: Lovász-Softmax

Rumus: Berbeda dari loss lain. Lovász-Softmax mengoptimasi **Jaccard index (IoU)** secara langsung. Rumusnya kompleks, tapi intuisinya:

- Hitung kesalahan per pixel (hinge loss: $\max(0, 1 - \text{margin})$).
- Urutkan kesalahan dari terbesar ke terkecil.
- Hitung Lovász extension: jumlahkan kesalahan dengan pembobotan berdasarkan peringkatnya.
- Ini adalah *surrogate* yang “konveks” untuk IoU.

Mengapa tidak kami pakai?

- Performa setara dengan Dice, tapi training 20% lebih lambat (perlu sorting per batch).
- Implementasi lebih kompleks, risiko bug lebih tinggi.
- Dice sudah cukup baik.

4.14 Ringkasan Loss Functions

Loss	Keunggulan	Kelemahan	Hasil
Dice	Sederhana, imbalance	Tidak smooth	Best
Tversky	Bobot FP/FN bisa diatur	Tuning α/β	Recall naik, P turun
Focal	Fokus hard samples	Tuning γ/α sulit	Di bawah Dice
Lovász	Optimasi IoU langsung	Lambat, kompleks	Setara Dice
Cross-Entropy	Standard, sederhana	Gagal total di imbalance	Collapse

5 Arsitektur: Kenapa Tiga Model?

5.1 Pertanyaan Fundamental

Untuk multi-temporal deforestation, ada **dua pertanyaan**:

1. Apakah menggunakan informasi pre-event *sama sekali* membantu dibandingkan hanya post-event?
2. Kalau ya, **bagaimana cara terbaik** menggabungkan pre dan post?

Untuk menjawabnya kita butuh 3 model:

5.2 Model 1: Single-Date (Baseline)

- Input: 5 channel (post-event only: R, G, B, NIR, NDVI).
- Arsitektur: SimpleUNet (ResNet34-style, 21.3M params).
- **Mengapa ini baseline?** Ini mewakili *status quo*: kebanyakan sistem deteksi deforestasi menggunakan satu citra saja. Lihat hutan, lihat bukan hutan. Tidak ada informasi perubahan.
- **Keterbatasan:** Tidak bisa membedakan “lahan yang selalu terbuka” dengan “lahan yang baru dibuka”. Kebun sawit, ladang kering, tanah kosong — semua terlihat mirip dengan deforestasi di satu titik waktu.

5.3 Model 2: Stacked Multi-Temporal

- Input: 10 channel (5 pre + 5 post, di-concatenate).

- Arsitektur: Sama dengan baseline, layer pertama diadaptasi dari 3 channel (pretrained) ke 10 channel.
- **Mengapa ada model ini?** Untuk menguji: apakah dengan memberikan informasi pre dan post *tanpa struktur khusus*, model bisa belajar perubahan?
- **Masalah:** Model menerima 10 channel, tapi tidak tahu mana pre dan mana post. Channel 1-5 pre, 6-10 post. Model harus belajar sendiri bahwa perbedaan channel 1-5 vs 6-10 itu signifikan. Ini tugas tambahan yang tidak perlu.

5.4 Model 3: Siamese FC-Siam-diff

- Input: Pre (5 channel) dan Post (5 channel) diproses **terpisah**.
- Arsitektur: Dua encoder ResNet34 dengan weight sharing. Decoder menerima $[f_{post}, f_{pre}, f_{post} - f_{pre}]$.
- **Inovasi:** Perbedaan fitur dihitung **secara eksplisit** di setiap level decoder. Model tidak perlu menebak mana yang berubah — perubahannya sudah disediakan.
- **Mengapa weight sharing?** Agar encoder menghasilkan representasi yang konsisten. Perbedaan output encoder semata-mata karena perbedaan input (pre vs post), bukan karena encoder berbeda.

5.5 Mengapa Bukan Model Lain?

- **Vision Transformer (ViT):** Butuh data sangat besar ($>100k$ images) untuk performa baik. Kita cuma punya 23k chips, dan itu pun 74% tanpa deforestasi.
- **DeepLabv3+:** Arsitektur kuat, tapi atrous convolution-nya (dilated convolution) boros memori untuk chip kecil 64×64 . Keuntungan ASPP (Atrous Spatial Pyramid Pooling) untuk multi-scale context kurang berguna pada chip kecil.
- **SegNet:** Lebih efisien memori (tanpa skip connections menyimpan banyak data), tapi akurasi boundary lebih rendah. Untuk deteksi deforestasi, boundary presisi itu penting.
- **Attention U-Net:** Menambahkan attention gate di skip connections. Ini interessante, tapi menambah kompleksitas. Prioritas kami adalah baseline multi-temporal dulu.

5.6 Pertanyaan Potensial di Sidang

- **Q:** Kenapa ResNet34 bukan ResNet50?
- **A:** ResNet50 (25.6M params) tidak muat di memori dengan batch 64. Selain itu, untuk chip 64×64 , representasi ResNet34 sudah cukup. Bottleneck ResNet50 (1×1 , 3×3 , 1×1) baru berguna untuk gambar lebih besar ($>224 \times 224$).
- **Q:** Bukannya weight sharing menghambat representasi spesifik?
- **A:** Weight sharing memaksa encoder mengekstrak fitur yang sama dari pre dan post. Justru ini yang kita mau: perbedaan deteksi harus berasal dari input, bukan dari encoder. Analogi: dua foto diambil kamera yang sama — perbedaan hasil karena subjeknya berubah, bukan karena kameranya berbeda.
- **Q:** Ada opsi arsitektur temporal lain?
- **A:** Banyak. ConvLSTM, 3D CNN, Temporal Attention Transformer. Tapi untuk 2 waktu (pre/post), Siamese sudah optimal. Arsitektur temporal kompleks baru berguna untuk time-series multi-waktu (>5 timestep).

6 Training: Semua Detail + Alasannya

6.1 Fungsi Loss: Kenapa Dice?

Pertanyaan inti: Dataset kita punya 0.38% pixel positif, 99.62% background. Loss function yang tidak dirancang untuk imbalance akan gagal total.

Kita coba jelaskan dengan analogi: Bayangkan kita mencari jarum di tumpukan jerami. Cross-entropy = ukur seberapa benar kita mengidentifikasi jerami. Dice loss = ukur seberapa baik kita menemukan jarum.

Matematika sederhana:

- **Cross-Entropy:** $CE = -[y \log(p) + (1 - y) \log(1 - p)]$. Untuk setiap pixel, hitung loss-nya. Background (99.62%) mendominasi total loss. Model optimal: prediksi semua 0 (background), CE rendah, IoU = 0.

- **Dice Loss:** $Dice = 1 - \frac{2TP}{2TP+FP+FN}$. Hanya pixel yang diprediksi positif berkontribusi. Tidak peduli background sebanyak apapun.

Uji coba loss lain:

- **Focal Loss:** Modifikasi CE yang menekan loss dari easy samples. Hasil: lebih buruk dari Dice. Kemungkinan karena hyperparameter γ dan α sulit dituning untuk extreme imbalance.
- **Tversky Loss:** Variasi Dice dengan bobot berbeda untuk FP dan FN. Dengan $\alpha = 0.7$ (bobot FN lebih besar), recall naik tapi precision turun drastis. Untuk aplikasi yang membutuhkan kepercayaan tinggi (alert system), precision juga penting.
- **Lovász-Softmax:** Optimasi langsung IoU. Hasil: setara dengan Dice tapi training lebih lambat (perlu sorting probabilities per batch).

6.2 Optimizer: Kenapa AdamW?

Adam (Adaptive Moment Estimation):

- Setiap parameter punya learning rate sendiri berdasarkan history gradient-nya. Cocok untuk masalah sparse gradient (banyak pixel background, sedikit foreground).
- Lebih stabil daripada SGD untuk loss landscape yang kompleks.

AdamW = Adam + Weight Decay yang benar:

- Weight decay = menambahkan $\lambda \cdot w$ ke gradient. Efek: parameter yang tidak berguna menuju nol (regularisasi).
- Di Adam biasa, weight decay tercampur dengan adaptive gradient — tidak efektif. AdamW memisahkan keduanya.
- Weight decay 5×10^{-4} : nilai standar untuk ResNet. Terlalu besar (10^{-2}): underfitting. Terlalu kecil (10^{-5}): tidak ada efek.

6.3 Learning Rate Scheduling: Kenapa Cosine Annealing?

Learning rate menentukan seberapa besar langkah update parameter.

- **Awal training:** LR tinggi (10^{-4}) = eksplorasi cepat. Model bergerak cepat di loss landscape.
 - **Akhir training:** LR rendah (menuju 0) = konvergensi halus. Model mencari minimum lokal yang baik.
- Cosine Annealing:** LR mengikuti kurva cosinus dari $10^{-4} \rightarrow 0$ selama 30 epoch. Transisi smooth.

Alternatif yang tidak dipilih:

- **Step decay** (turun di epoch 10, 20): Penurunan mendadak bisa mengganggu konvergensi.
- **Reduce on plateau** (turun kalau val loss stagnan): Butuh patience tuning. Untuk training dengan 30 epoch, tidak cukup waktu untuk plateau detection.
- **One-cycle:** Menaikkan LR dulu baru turun. Bagus untuk fast training tapi butuh tuning max LR.

6.4 Gradient Accumulation: Kenapa Accum=2?

Masalah: RTX 3060 hanya muat batch size 64. Tapi batch 64 terlalu kecil untuk training stabil dengan class imbalance.

Solusi: Gradient accumulation. Proses:

1. Forward pass batch 1 (64 chips), hitung loss, hitung gradient. **Tapi jangan update parameter.**
2. Forward pass batch 2 (64 chips), hitung loss, akumulasi gradient. **Baru update parameter.**

Efek: Gradient dari 128 chips diakumulasi sebelum update. Sama seperti batch size 128, tapi tanpa perlu memori $2\times$.

Mengapa accumulation 2, bukan 4 atau 8? Accum 2 sudah cukup untuk mencegah collapse. Accum 4 (efektif 256) tidak memberikan peningkatan signifikan. Accum 8 tidak muat di memori (perlu nyimpen activation untuk 512 chips).

6.5 Oversampling: Kenapa $10\times$?

Masalah: 74% chips tanpa deforestasi. Sampling acak: rata-rata hanya 17 dari 64 chips per batch mengandung deforestasi.

Oversampling: Setiap chip positif di-sampling $10\times$ lebih sering. Cara: dataset dibagi jadi positif (26%) dan negatif (74%). Dari positif, sampling dengan replacement $10\times$ ukuran asli. Gabung dengan negatif, shuffle.

Efek: Proporsi chips positif dalam batch $\sim 50\%$. Model cukup sering melihat deforestasi untuk belajar.

Mengapa 10, bukan 5 atau 20?

- **$5\times$:** Cuma 30% batch positif. Masih terlalu sedikit — model tetap bias ke background.

- **20×**: Dominasi chips positif yang sama (di-sampling berulang). Model overfit ke beberapa contoh spesifik.
- **10×**: Proporsi $\sim 50:50$. Ideal.

6.6 Augmentasi: Kenapa Minimal?

Data augmentation = memvariasikan data training secara artifisial. Tujuan: meningkatkan generalisasi.

Kami hanya menggunakan flip horizontal + vertikal. Mengapa?

- **Spectral consistency sangat penting.** Deforestasi dideteksi dari perubahan spektral (kandungan NIR turun, NDVI turun). Rotasi mengubah sudut datang cahaya, brightness mengubah intensitas, Gaussian noise mengubah distribusi spektral. Semua ini merusak sinyal deforestasi.
- **Definisi "berubah" itu absolut.** Kalau kita mengubah kecerahan pre dan post secara independen, selisih NDVI sebelum-sesudah juga berubah — padahal di lapangan tidak ada perubahan kecerahan.
- **Eksperimen validasi:** Heavy augmentation (rotation $\pm 30^\circ$, brightness $\pm 10\%$, noise $\sigma = 0.01$) menurunkan val IoU dari 0.044 ke 0.031. Penurunan 30%.

Flip aman: Deforestasi tidak memiliki orientasi — hutan yang dibuka dari utara atau selatan tetap deforestasi. Flip tidak mengubah nilai spektral.

6.7 AMP (Mixed Precision): Kenapa?

FP32 vs FP16:

- FP32: 32-bit floating point. Presisi tinggi, memori besar.
- FP16: 16-bit floating point. Presisi setengah, memori setengah, throughput $2\times$ di Tensor Cores.

AMP = Automatic Mixed Precision. Bagian kritis (loss computation, gradient scaling) tetap FP32. Bagian komputasi berat (konvolusi) di FP16.

Keuntungan untuk RTX 3060:

- Tensor Cores (generasi Ampere) memberikan $2-3\times$ throughput di FP16 vs FP32.
- Memory usage turun 30% — memungkinkan batch 64 atau model dengan lebih banyak channel.
- Loss scaling mencegah underflow gradient (gradient terlalu kecil untuk FP16).

6.8 Gradient Clipping: Kenapa 1.0?

Problem: Dengan Dice loss dan imbalance, kadang ada batch yang loss-nya sangat tinggi (misal batch berisi deforestasi yang sulit). Gradient-nya bisa sangat besar, menyebabkan parameter update yang ekstrem — merusak representasi.

Solusi: Clip gradient norm ke 1.0. Artinya: kalau panjang vektor gradient > 1.0 , skala ulang ke 1.0. Informasi arah gradient tetap, besarnya dibatasi.

Kenapa 1.0? Terlalu kecil (0.1) menghambat learning. Terlalu besar (10.0) tidak efektif. 1.0 adalah nilai default yang umum.

6.9 RAM Caching: Kenapa?

Tanpa cache: Setiap epoch, 23,135 chips dibaca dari SSD. Setiap chip: load file NPZ ($10 \text{ bands} \times 64 \times 64$), parse, stack ke tensor. Total: ~ 210 detik/epoch. 30 epoch = 105 menit hanya I/O.

Dengan cache: Epoch 1: load semua ke RAM (24 GB). Epoch 2-30: baca dari RAM. ~ 27 detik/epoch. 30 epoch = 16 menit.

Mengapa feasible? $23,135 \text{ chips} \times 10 \text{ bands} \times 64 \times 64 \times 4 \text{ bytes} = \sim 3.8 \text{ GB}$. Komputer punya 32 GB RAM, 24 GB free setelah OS. Muat.

7 Inference & Evaluation

7.1 Test-Time Augmentation (TTA)

Ide: Saat inference, jangan hanya jalankan sekali. Jalankan 3 kali (original, flip horizontal, flip vertical), rata-rata probabilitasnya.

Kenapa efektif? Ensemble 3 model (dengan parameter sama tapi input berbeda). Setiap varian flip punya "kesalahan" berbeda. Rata-rata menghaluskan kesalahan.

Hasil:

- Single-date: IoU $0.119 \rightarrow 0.122$ (+2.5%).

- Siamese: IoU 0.132 $\rightarrow$ 0.137 (+3.8%).

7.2 Global vs Per-Sample Metrics

Ini **sangat penting** untuk dipahami. Dua cara menghitung IoU:

Per-sample mean:

1. Hitung IoU tiap chip: $IoU_1, IoU_2, \dots, IoU_n$.
2. Rata-rata: $\frac{1}{n} \sum IoU_i$.
3. **Masalah:** 74% chips IoU=0 (tanpa deforestasi). Mean IoU $\sim$ 0.013. Angka ini sangat rendah dan tidak informatif.

Global (summed):

1. Jumlahkan semua TP, FP, FN dari semua chips.
2. $IoU_{global} = \frac{\sum TP}{\sum TP + \sum FP + \sum FN}$.
3. **Keuntungan:** Chips tanpa deforestasi tidak berkontribusi (TP=0, FP=0, FN=0). Hanya chips dengan deforestasi yang memengaruhi metrik.

Kapan pakai yang mana? Per-sample: kalau kita peduli per-chip (setiap chip harus benar). Global: kalau kita peduli total area (seberapa luas deforestasi terdeteksi). Untuk aplikasi monitoring, global lebih relevan.

7.3 Mengapa Threshold 0.5?

Dice loss secara implisit mengoptimalkan threshold 0.5. Model dilatih untuk output probabilitas, dan dice loss optimal saat threshold 0.5. Kalau kita ubah threshold (misal 0.3), precision turun, recall naik — trade-off. 0.5 adalah titik natural.

7.4 Pertanyaan Potensial

- **Q:** *Kenapa validation IoU lebih rendah dari test IoU?*
- **A:** Distribusi deforestasi tidak seragam. Test set memiliki proporsi deforestasi lebih tinggi (0.9%) daripada validation set (0.38%). Karena metrik global dihitung dari total TP/FP/FN, semakin banyak deforestasi sebenarnya yang ada, semakin tinggi IoU-nya (selama model tidak all-background). Ini menunjukkan bahwa model bekerja lebih baik di area dengan deforestasi lebih padat.
- **Q:** *Apakah metrik global tidak menyesatkan? Kalau satu chip mendominasi, metrik bisa bias.*
- **A:** Valid. Itu sebabnya kami juga melaporkan per-sample mean. Tapi untuk tujuan deteksi area, chip besar dengan banyak deforestasi harusnya berbobot lebih — karena area deforestasi yang lebih luas lebih penting untuk dideteksi. Global metrics mencerminkan ini.

8 Hasil: Interpretasi Lengkap

8.1 Tabel Hasil

Model	IoU	Precision	Recall	F1
Single-date (post only)	0.122	0.198	0.239	0.217
Multi-temporal (stacked)	0.119	0.154	0.344	0.212
Siamese (ours)	0.137	0.196	0.311	0.240

8.2 Analisis Per Model

Single-date (post only):

- Precision tertinggi (0.198). Artinya: dari semua prediksi positif, 19.8% benar.
- Recall terendah (0.239). Hanya 23.9% deforestasi sebenarnya yang terdeteksi.
- **Interpretasi:** Model ini konservatif. Hanya memprediksi positif kalau sangat yakin. Akibatnya, banyak deforestasi terlewat.
- **Kenapa?** Karena tidak punya informasi "sebelumnya", model harus menebak dari post-event saja. Area yang spektralnya mirip deforestasi (lahan terbuka, pertanian) tidak bisa dibedakan.

Stacked multi-temporal:

- Recall tertinggi (0.344). Mendeteksi 34.4% deforestasi.
- Precision terendah (0.154). Banyak false positive.
- **Interpretasi:** Model ini agresif. Concatenation 10 channel memberi informasi pre+post, tapi model tidak bisa memanfaatkannya dengan baik. Akibatnya: perubahan spektral *apapun* dianggap deforestasi — termasuk perubahan musiman (pertanian, air).
- **Kenapa stacking kalah?** Informasi "mana channel pre dan post" hanya dari urutan. Model harus belajar relasi antar channel 1-5 vs 6-10. Ini tugas tambahan yang tidak perlu. Siamese memberikan relasi ini secara eksplisit.

Siamese FC-Siam-diff:

- IoU tertinggi (0.137), F1 tertinggi (0.240).
- Precision (0.196) hampir menyamai single-date.
- Recall (0.311) mendekati stacked.
- **Interpretasi:** *Best of both worlds*. Explicit difference features memungkinkan model mendeteksi perubahan dengan presisi tinggi (seperti single-date) dan recall baik (seperti multi-temporal).
- **Kenapa siamese menang?** Karena model tidak perlu menebak:
 1. Encoder weight sharing: representasi konsisten.
 2. Decoder menerima $f_{post} - f_{pre}$: perubahan langsung diberikan, bukan disembunyikan.
 3. Skip connections dengan difference: perubahan di berbagai skala (dari detail tepi hingga fitur global).

8.3 Mengapa IoU Hanya 0.137?

Pertanyaan paling berat. Jawaban jujur:

1. **Label noise (faktor dominan):** GFW label 30 m, citra 10 m. Satu pixel GFW = 9 pixel S2. Misal: deforestasi 0.5 ha di GFW — area 50×50 m — direpresentasikan sebagai ~ 3 pixel Landsat (masing-masing 30×30 m). Di S2 (10m), ini area 50×50 pixel. Tapi labelnya hanya 3 pixel. Model bingung: prediksi besar, label kecil. Ini dihitung sebagai FP + FN, menurunkan IoU.
2. **Class imbalance:** 0.38% positif. Dari 1000 pixel, hanya 4 yang deforestasi. Model bisa prediksi salah 2 dan masih akurasi 99.8%. Tapi IoU-nya anjlok.
3. **74% chips tanpa deforestasi:** Ini masalah untuk training (pengaruh ke loss) dan untuk evaluasi (pengaruh ke per-sample metrics).
4. **Chip size 64×64 :** Terlalu kecil untuk konteks landscape. Model tidak bisa membedakan "ini tepi hutan" dari "ini deforestasi baru".
5. **Cloud-free composite:** Deforestasi yang terjadi di pertengahan tahun ter-blur dalam komposit.

8.4 Perbandingan dengan ICP Paper (NDVI pseudo-label)

ICP paper (sebelumnya) mencapai IoU 0.73. **Mengapa jauh berbeda?**

- Label ICP = NDVI differencing: $NDVI_{post} - NDVI_{pre}$. Threshold untuk menentukan deforestasi. Ini adalah **metode hand-crafted** — label dibuat dari data yang sama dengan input. Model U-Net belajar meniru decision boundary yang sudah ada. Wajar IoU tinggi.
- GFW label = **independent source** dari Landsat. Model harus belajar korelasi antara citra S2 10 m dan label L8 30 m. Ini tugas yang jauh lebih sulit.
- ICP IoU 0.73 = validasi internal (NDVI vs NDVI). GFW IoU 0.137 = validasi eksternal (S2 vs Landsat). Dua hal berbeda.

8.5 Overfitting: Analisis

Gejala: Val IoU peak di epoch 4-6, lalu turun. Training loss terus turun (model menghafal data training). Validation loss naik atau plateau.

Penyebab:

- **Rasio parameter:sampel:** 21.3M parameter, 23k training chips (efektif hanya $\sim 6k$ yang positif). Rasio 3,550:1. Idealnya $< 100:1$.
- **Dataset homogen:** 13 scene dari 2 provinsi. Variasi terbatas — model cepat "bosan" dan mulai menghafal.
- **Regularisasi terbatas:** Weight decay $5e-4$ saja. Dropout akan membantu, tapi menambah waktu training dan bisa menurunkan peak performance.

Solusi potensial (untuk sidang/future work):

- **More data:** Tambah scene dari provinsi lain.
- **Label smoothing:** Mengurangi confidence model terhadap label noise.

- **Dropout:** Di decoder (biasanya 0.2-0.5).
- **Early stopping:** Berhenti di epoch 5-6 (peak val IoU). Tidak perlu 30 epoch penuh.

9 Pipeline: Gambaran Sistem End-to-End

1. **Data Export (GEE):** Google Earth Engine. Export Sentinel-2 composite (2020, 2023) + GFW lossyear layer sebagai GeoTIFF.
2. **Chip Generation:** Tile GeoTIFF ke 64×64 chips. Pair pre/post. Simpan sebagai NPZ (5 bands + mask).
3. **Training:** Load chips ke RAM. Oversampling. DataLoader dengan augmentasi flip. Model forward/backward dengan AMP + gradient accumulation. Log metrics ke CSV.
4. **Inference:** Load chip test. TTA (3 flips). Threshold 0.5. Hitung metrics (global + per-sample). Simpan visualisasi (6-panel plot).
5. **Analisis:** Bandingkan 3 model. Plot kurva training. Analisis error.

Hardware:

- GPU: NVIDIA RTX 3060 (12GB VRAM).
- CPU: Intel/AMD (cukup untuk data loading).
- RAM: 32 GB (24 GB untuk cache).
- Storage: SSD (I/O bottleneck tanpa cache).
- OS: Windows 11 + WSL2 (Ubuntu).

Software:

- PyTorch 2.7, CUDA 12.x.
- AMP (torch.cuda.amp).
- Rich (progress bar TUI).
- tmux (survive SSH disconnect).

Performance:

- Training 30 epochs: ~15 menit (dengan cache).
- Inference 11,527 chips + TTA: ~12 menit.
- GPU memory usage: ~8 GB (peak).

10 Pertanyaan Sulit yang Mungkin Muncul

10.1 Tentang Relevansi dan Kontribusi

- **Q:** *IoU 0.137 — ini sangat rendah. Apa gunanya?*
- **A:** Ini *proof of concept*. Tujuan kami bukan mencapai akurasi operasional, tapi membuktikan pipeline end-to-end bisa berjalan dengan label yang ada. IoU 0.137 dengan GFW label noise adalah baseline. Dengan label manual (yang merupakan langkah logis berikutnya), IoU diperkirakan meningkat 3-5×. Nilai dari proyek ini adalah: **(1)** pipeline dari GEE ke visualisasi, **(2)** perbandingan arsitektur, **(3)** identifikasi bottleneck (label noise adalah masalah utama).
- **Q:** *Apa bedanya dengan pekerjaan yang sudah ada?*
- **A:** Kebanyakan penelitian deforestasi menggunakan label NDVI pseudo-label (seperti ICP paper kami sebelumnya) atau label manual. Yang **unik** dari proyek kami: menggunakan GFW label — yang available secara global — untuk melatih model 10 m. Ini menunjukkan bahwa GFW, meskipun tidak sempurna, bisa menjadi *weak supervision* untuk deteksi resolusi lebih tinggi. Kontribusi: **metode** dan **pipeline**, bukan absolute accuracy.
- **Q:** *Kenapa tidak langsung pake NDVI differencing aja? Itu IoU-nya 0.73.*
- **A:** NDVI differencing adalah metode *rules-based*: threshold manual yang mungkin tidak generalizable ke lokasi lain. Deep learning bisa belajar pola yang lebih kompleks dari data multi-band, bukan hanya NDVI. Selain itu, proyek ini adalah eksperimen: *can we leverage existing GFW labels?* Jawabannya: yes, dengan batasan tertentu.

10.2 Tentang Metode

- **Q:** *Mengapa tidak pakai data time-series (multi-temporal > 2 waktu)?*

- **A:** Terbatas oleh GFW label — hanya tahunan. Kalau kita punya 6 time points (2018-2023), labelnya cuma satu: "deforestasi terjadi di tahun X". Tidak ada label per-waktu. Model tidak bisa belajar temporal dynamics tanpa temporal labels. Arsitektur 2-waktu (pre/post) adalah yang paling sesuai dengan data yang ada.
- **Q:** *Bukankah lebih baik pakai unsupervised change detection?*
- **A:** Unsupervised (seperti PCA, MAD, IR-MAD) tidak memerlukan label. Tapi outputnya anomaly score — bukan "deforestasi" secara semantik. Tidak bisa membedakan deforestasi dari perubahan musiman, banjir, atau kebakaran. Supervised learning memberikan *semantic understanding*: model belajar *apa itu* deforestasi, bukan hanya *apa yang berubah*.
- **Q:** *Ukuran chip 64×64 itu terlalu kecil. Kenapa tidak 256×256?*
- **A:** Terbatas GPU. Tapi yang lebih penting: eksperimen dengan chip lebih besar membutuhkan re-export data dari GEE + re-training + re-inference — siklus yang panjang. Dengan 64×64, siklus eksperimen cepat (~15 menit per training). Ini memungkinkan hyperparameter search yang ekstensif (9 konfigurasi). Setelah arsitektur final dipilih, scaling ke chip lebih besar adalah engineering work, bukan research.

10.3 Tentang Evaluasi

- **Q:** *Kenapa tidak pake Kappa atau AUC?*
- **A:** Kappa: sensitive terhadap prevalence (proporsi kelas). Dengan 0.38% prevalence, Kappa bisa tinggi meski IoU rendah. AUC (ROC): untuk binary classification dengan threshold. Segmentasi adalah pixel-level, IoU adalah standar de facto untuk segmentation task. Precision/Recall/F1 melengkapi IoU untuk interpretasi yang lebih nuanced.
- **Q:** *Apakah hasil ini reproducible?*
- **A:** Ya. Semua kode ada di GitHub (public). Semua hyperparameter dicatat. Random seed di-set. Satu-satunya yang tidak fully reproducible: cloud-free composite dari GEE (algoritma median bisa sedikit berbeda tergantung versi). Tapi secara umum, hasil harus konsisten dalam ± 0.01 IoU.

10.4 Tentang Aplikasi

- **Q:** *Apa rencana implementasi ke depannya?*
- **A:** (1) Perbaiki label: manual annotation atau Planet imagery (3m) untuk label lebih presisi. (2) Scale: 256×256 chips, model lebih besar. (3) Operationalize: deploy sebagai API yang menerima citra baru dan menghasilkan alert. (4) Integrasi SAR Sentinel-1 untuk kemampuan all-weather.
- **Q:** *Apakah model bisa dipakai di daerah lain?*
- **A:** Belum diuji. Model dilatih di Riau dan Kalteng. Generalisasi ke Papua (tipe hutan berbeda), Sumatera Barat (topografi berbeda), atau Jawa (fragmentasi tinggi) perlu diuji. Tapi karena arsitekturnya general-purpose, kami optimis dengan fine-tuning atau tambahan data dari daerah target.

11 Limitations: Jawaban Jujur

1. **Label noise:** Ini batasan **fundamental**. Selama kita pakai GFW label 30 m, akurasi maksimal terbatas. Model tidak bisa lebih presisi dari labelnya.
2. **Chip size:** 64×64 membatasi konteks spasial. Tapi trade-off dengan GPU memory.
3. **Temporal coverage:** Dua waktu (pre/post) dengan gap 3 tahun. Deforestasi di 2021 dan 2022 tercampur. Idealnya: tahunan atau sub-tahunan.
4. **Geographic scope:** Hanya 13 scene dari 2 provinsi. Belum mewakili keragaman hutan Indonesia.
5. **False positives:** Perubahan non-deforestasi (pertanian, badan air musiman, kebakaran) masih menghasilkan false positives.
6. **Cloud dependence:** Sentinel-2 optical — tidak bisa menembus awan. Di musim hujan, bisa berbulan-bulan tanpa citra bersih.

12 Future Work: Arah Pengembangan

1. **Better labels:** Ini prioritas nomor satu. 500-1000 chips dengan label manual di resolusi 10 m akan memberikan lompatan kualitas. Bisa juga pakai Planet imagery (3 m) untuk anotasi.
2. **Larger chips:** 128×128 atau 256×256 dengan gradient accumulation lebih besar. Memberikan konteks landscape yang lebih baik.
3. **SAR integration:** Sentinel-1 (radar) + Sentinel-2 (optical). Radar tidak terpengaruh awan. Kombinasi optical + radar memberikan coverage temporal yang jauh lebih baik.
4. **Multi-temporal time-series:** 5+ time points dengan temporal attention atau ConvLSTM. Bisa mendeteksi *kapan* deforestasi terjadi, bukan hanya *apakah*.
5. **Post-processing:** Connected-component filtering (buang prediksi < 0.01 ha), CRF untuk boundary smoothing.
6. **Deployment:** API endpoint + dashboard monitoring. Integrasi dengan Google Earth Engine untuk alert near-real-time.
7. **Uncertainty estimation:** Monte Carlo Dropout atau Deep Ensemble untuk memberikan confidence score pada setiap prediksi.

13 Quick Reference Cards

Arsitektur Comparison

Aspek	Single-date	Stacked	Siamese
Input channels	5 (post)	10 (pre+post)	5+5 (terpisah)
Parameters	21.3M	21.3M	25.9M
Fusion method	N/A	Concatenation	Explicit diff
Pretrained	Adapted	Adapted	Shared
Weight sharing	N/A	N/A	Yes
Best IoU	0.122	0.119	0.137
Key strength	Precision	Recall	Balance
Key weakness	Low recall	Low precision	Complexity

Hyperparameter Summary

Hyperparameter	Nilai	Alasan
Loss function	Dice (symmetric)	0.38% imbalance
Optimizer	AdamW	Adaptive + weight decay
Learning rate	10^{-4}	Default Adam
Weight decay	5×10^{-4}	Regularisasi ringan
Scheduler	Cosine annealing	Smooth decay
Batch size	64 (eff. 128)	GPU memory
Accumulation	2	Simulasi batch 128
Oversampling	$10 \times$	74% zero-deforest chips
Augmentation	H/V flip only	Spectral consistency
Precision	AMP (FP16)	Tensor Cores speed
Gradient clipping	1.0	Prevent gradient spikes
Epochs	30	Overfitting after 6

Defense Script — 60 Second Answers

- **Apa yang kamu buat?** Sistem deteksi deforestasi otomatis dengan deep learning pada Sentinel-2 10 m, menggunakan label GFW 30 m sebagai training target.
- **Kenapa tiga model?** Untuk menguji apakah multi-temporal fusion membantu, dan arsitektur mana yang terbaik.
- **Kenapa IoU 0.137 rendah?** Label noise GFW (30m vs 10m), extreme class imbalance (0.38% positif), dan 74% chips tanpa deforestasi.
- **Mengapa siamese menang?** Karena explicit difference features memberikan sinyal perubahan yang lebih bersih daripada concatenation.

- **Apa kontribusi utama?** Pipeline end-to-end dari GEE ke trained model + perbandingan arsitektur + identifikasi label noise sebagai bottleneck utama.
- **Langkah selanjutnya?** Label manual, chip lebih besar, SAR integration.

References

- [1] M. C. Hansen, P. V. Potapov, R. Moore, et al. High-resolution global maps of 21st-century forest cover change. *Science*, 342(6160):850–853, 2013.
- [2] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 9351:234–241, 2015.
- [3] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [4] R. C. Daudt, B. Le Saux, and A. Boulch. Fully convolutional siamese networks for change detection. In *IEEE International Conference on Image Processing (ICIP)*, pages 4063–4067, 2018.